

effortlessly.

Purpose

This document defines the product behavior and user-value layer that sits above UI. Agents must preserve these principles even when changing layouts, components, routes, or implementation details.

North Star: A group should be able to answer “What are we doing?”, “Who is involved?”, “What is booked?”, “What do I owe?”, and “What happens next?” without reconstructing the plan from chat messages and payment screenshots.

Non-negotiable product promise

- One shared plan is the source of truth for the group.
- People, decisions, bookings, expenses and settlement are connected contextually.
- The system surfaces next actions instead of making users hunt through modules.
- The experience feels social and human, not like an accounting or project-management tool.

Anti-pattern: If a feature adds data or screens but does not reduce coordination friction, it is not automatically valuable.

01 - The user problem

The real problem is not “booking”. It is fragmented coordination. The same group repeatedly has to reconstruct context across chat, booking apps, email, screenshots and payment tools.

Typical chaos

Idea	People	Decision	Booking	Money	Follow-up
“Go Goa?”	Who is coming?	Which day?	Who booked?	Who paid?	What is pending?

Product response

BookKaroYaar should compress the conversation-to-action loop:

Question → decision → shared plan → booking → expense → settlement → done.

Do not use fake statistics

Marketing and UI copy may dramatize the problem through examples, but must not invent precise user numbers or claims unless they come from an approved source. The product is compelling because of the workflow, not fabricated proof.

02 - Core product pillars

These are the capabilities that make BookKaroYaar meaningfully different. They should remain visible across the product, even as UI evolves.

01

ONE-PLAN WORKSPACE

Every trip, movie night, dinner or event has one contextual home. The plan is the container for people, itinerary, bookings, expenses and activity.

02

PEOPLE-FIRST COORDINATION

Membership is not just metadata. The product continuously answers who is in, who voted, who booked, and who still needs to respond.

03

DECISION → ACTION

Voting and planning should not end in a poll. Once the group decides, the decision should feed the next action: create plan, booking, task or itinerary item.

04

BOOKING ↔ EXPENSE

A booking and its money should be connected. If a person books a hotel for four, the expense flow should already understand participants and payer context.

05

MONEY WITHOUT SPREADSHEETS

Equal, percentage and custom splitting produce understandable shares and balances without manual arithmetic.

06

SMART SETTLEMENT

The system turns net balances into clear actions: who should pay whom, how much, and what remains pending.

07

WHAT NEEDS ATTENTION?

The app proactively surfaces the next useful action instead of forcing users to inspect every section.

03 - The core experience loop

Every primary interaction should reinforce a single loop. This is the conceptual backbone for future modules and UI decisions.

01 - START

Create a plan from an intention: trip, movie, dinner, event, or other.

02 - BRING PEOPLE

Invite people via shareable link or existing group membership.

03 - DECIDE

Use votes and lightweight choices when the group needs alignment.

04 - PLAN

Build dates, itinerary items, notes and responsibilities.

05 - BOOK

Record or connect bookings inside the plan.

06 - SPLIT

Turn shared spending into participant shares automatically.

07 - SETTLE

Show net balances and simplify who pays whom.

08 - DONE

Close the plan with history intact, so nothing has to be reconstructed later.

Design implication: Navigation and dashboards should make this loop visible. A user should feel that every feature advances a plan rather than creating another isolated record.

04 - The command-center experience

The dashboard is not primarily a statistics page. Its job is to reduce uncertainty and reveal the next useful action.

The dashboard should answer five questions

What am I planning?

Show active and upcoming plans with strong contextual identity.

Who is involved?

Show people, participation and outstanding responses.

What happened?

Surface meaningful recent activity, not every database mutation.

What do I need to do?

Prioritize actions such as vote, confirm, pay, review or finish setup.

Where do I stand with money?

Show owed/owing/settled in human language.

Example product language

“2 things need your attention” is stronger than “2 pending items”. “You owe Rahul ■800” is stronger than “Outstanding balance ■800”. Use human action language.

05 - Features must connect

No major feature should feel like an island. The value comes from the connections between objects.

Feature	Should naturally lead to	User value
Poll	Plan / decision	The group decision becomes actionable
Invite	People / membership	The right people enter the shared context
Booking	Expense	Money context is captured at the moment of purchase
Expense	Settlement	No manual reconciliation
Settlement	Plan / activity	The group can see what is still open
Activity	Attention	The system surfaces meaningful next actions

Agent rule: When adding a new feature, identify what existing user problem it connects to and which object it updates. Avoid disconnected CRUD screens.

06 - UX and implementation guardrails

Product polish is not only visual. The system should behave predictably under real collaboration.

Required behavior

- Loading state before remote data appears.
- Empty state that explains what the user can do next.
- Error state with recovery or retry when possible.
- Optimistic update for suitable low-risk interactions.
- Realtime updates where group collaboration benefits from them.
- Clear success confirmation for important actions.
- Permission-aware UI, with Firebase Security Rules as the actual security boundary.

Do not

- Create generic feature dashboards with no next action.
- Duplicate data entry that the app already knows.
- Make users manually recalculate group expenses.
- Hide important state inside menus when it could be visible contextually.
- Replace the product story with visual decoration.
- Invent product facts, user numbers or fake analytics.

Visual principle: The app can be playful and expressive, but every playful element must support clarity. “Funky” is a treatment, not an excuse for friction.

07 - Agent acceptance criteria

Before an agent declares a module complete, it must be possible to explain how that module reduces a specific coordination problem.

- Identify the user problem the module solves.
- Identify the workflow it shortens or makes clearer.
- Connect the feature to the relevant plan/group objects.
- Preserve the core product loop: people → plan → booking → money → settlement.
- Show the next useful action where appropriate.
- Handle loading, empty, error, success and permission states.
- Keep the visual design consistent with the BookKaroYaar visual source of truth.
- Update DEVELOPMENT_STATE.md after meaningful milestones.
- Do not invent major product behavior outside the specifications.

Feature review question

Before implementation is considered complete, ask: **“If we remove this feature, what user problem gets worse?”** If the answer is unclear, reconsider the feature scope or its placement.

Current roadmap impact

Module 02 - App Shell + Dashboard should be implemented as the “Command Center” for the core experience, not as a generic dashboard. The next modules should deepen the loop rather than create isolated CRUD utilities.